

BIRMINGHAM CITY UNIVERSITY

Faculty of Computing, Engineering and the Built Environment

CMP7200 | Individual Master's Project | Assessment 1: Project Proposal

AI Trip Planner

*A Multi-Agent Travel Planning System Built on the
Model Context Protocol and a Custom Agent-to-Agent Protocol*

Student Number	Shahwaiz Ahmad
Student ID	25182589
Module	CMP7200 Individual Master's Project
Module Leader	Samer Bamansoor
Submission Date	22 June 2026, 3 PM (via Moodle)

*Submitted in partial fulfilment of the requirements for the degree of
Master of Science in Artificial Intelligence*

Abstract

Short trips are difficult to plan, no matter how short. When the user says “a week in Bangkok for \$2,000”, its meaning is ambiguous, it requires the assistant to find up-to-the-minute flight and hotel information, it demands a particular price budget, and it expects the assistant to come up with a plan that makes sense when it is fed back to the real world for verification. With the current single-LLM assistants, numbers for flights are often just made up, and hotel recommendations are for non-existing hotels (Xie et al., 2024). This project goes from another perspective. Each agent specializes in a specific aspect of the problem, and the work has two well-defined protocols: the Model Context Protocol (MCP) (Anthropic, 2024) for tool calls, and a custom Agent-to-Agent (A2A) protocol for typed inter-agent messages. The platform is composed of three live APIs (Kiwi.com, Booking.com and Serper.dev) that are behind a single MCP server, and a Streamlit interface that allows the user to talk to the API in natural language. The proposal places the work in the context of the literature, lays out a design-science methodology and provides an 11-week schedule from 23 June to 7 September 2026.

1. Background and Rationale

1.1 Motivation

Tourism is among the planet's biggest service sectors, and international receipts of approximately USD 1.9 trillion in 2023 (UNWTO, 2024). For the average consumer, however, the planning process has remained more or less comparable over the past 20 years. People still change from 5 to 10 websites and are constantly juggling browser tabs and re-constructing the plan every time a price changes. My word with friends and a quick scan through the travel-forum threads I ran while looking at this project shows a consistent picture: "It's a good trip; it's a tedious trip to plan. The value of a well-designed automation of the planning side would be meaningful for a very large number of users of an everyday task.

1.2 Why current AI assistants fall short

The hope might be that GPT-4o-type models have already solved this issue. They have not. At the Travel Planner benchmark, the best single-LLM agent achieves just 0.6% final-pass rate (Xie et al., 2024) with most errors being either due to an inconsistent date or due to venues being hallucinated or a violation of the budget constraint. I find three themes from the literature. First there is context bloat - a single Booking.com hotel response can be 8-12kB of raw JSON - everything is pushed out of the way to accommodate the previous reasoning. The second is protocol fragility: The model generates invalid tool parameters (Schick et al., 2023). Third, semantic drift: Free-text instructions handed over between cooperating agents get corrupted at each handoff (Park et al., 2023). These three failure modes combined make up the design goal for this project.

1.3 Aim

To design, construct and test a multi-agent travel planning system that generates verifiable, bookable, day-by-day travel plans from natural language requests that integrates a schema-validated MCP tool layer over three real travel APIs and a typed agent-to-agent messaging protocol.

1.4 Objectives

The aim is subdivided into the following objectives, all of which have one or more learning outcomes (LO1–LO5) associated with them.

1. Develop a custom Python based MCP server that serves three live travel APIs (Kiwi.com, Booking.com, Serper.dev) from a total of 13 schema-validated tools (LO3, LO4).
2. Create and design an A2A protocol with 6 types of messages and priority queuing and conversation tracking (LO3, LO4).
3. Use a heterogeneous mix of GPT-4o and GPT-4o-mini to implement the 6 CrewAI agents: Conversational, Preferences Extractor, Flight, Hotel, Attraction and Itinerary Coordinator to control the cost of the implementation (LO2/LO4).
4. Use a thread-pool to execute the three search agents simultaneously, before and after the budget and time constraints (LO4).
5. Create a Streamlit web app to demonstrate live in viva (LO4).
6. Perform ablation studies measuring the impact of MCP, A2A and parallelism with respect to the single-LLM baseline (LO2, LO5).
7. Compare and contrast the cost, latency and bookability in at least twenty scenarios, noting limitations (LO1, LO5).

1.5 Scope

In scope	Out of scope (future work)
English conversational input	Multilingual NLP
Three APIs: Kiwi, Booking, Serper	GDS / Sabre / Amadeus
Thirteen MCP tools	Booking transactions and payments
Six CrewAI agents	User accounts / login
Streamlit web UI	Native mobile applications
Retrospective evaluation, 20 scenarios	Live user study (no ethics approval)

Table 1. Project scope.

1.6 Expected contributions

This should result in three deliverables: a simple implementation (in the open-source spirit) of how the combination of MCP and a typed A2A envelope can be realized in a multi-agent system using live consumer APIs; an ablation study, measuring the gains of each protocol against a single-LLM baseline on this domain; and a working Streamlit demo and clean GitHub repo that can be used during the viva.

2. Short Literature Review

This review is a survey of four threads of recent work, all of which are briefly outlined, and a small conceptual model is provided which connects the threads. The references are in the style of BCU Harvard.

2.1 Single-LLM tool use

In the first wave of LLM-plus-tools, it was found that one model can make the decision to call an external function. To self-supervise a model for inserting API calls into its generation, Toolformer (Schick et al., 2023) is proposed. React (Yao et al., 2023) interleaves reasoning steps with actions in a loop, and Reflexion (Shinn et al., 2023) introduces self-critique. When there are numerous tool calls, this approach has poor scalability because context windows are filling up and early instructions are being pushed away by successive observations: With multi-city traveling tasks, one copy of GPT-4 alone fails to adhere to basic temporal constraints in approximately 31% of its plans (Xie et al., 2024).

2.2 Multi-agent frameworks

A number of frameworks have emerged that orchestrate agents with the LLM specialization. Natural-language conversation between agents is used in AutoGen (Wu et al., 2023). Each role is assigned a standard operating procedure in MetaGPT (Hong et al., 2024). The GPTSwarm (Zhuge et al., 2024) is an optimization of agent topologies that is based on graphs. CrewAI (CrewAI, 2024) adopts a simpler approach with roles/goal/Tools. Two recent surveys (Xi et al., 2023; Wang et al., 2024) provide helpful background. Before I decided on a framework I did a comparison in Table 2.

Framework	Messaging style	Native MCP	Streamlit fit	Decision
CrewAI	Role / goal / tools	Yes	Excellent	Selected
AutoGen	Free-form conversation	Indirect	Possible	Rejected
MetaGPT	SOPs per role	No	Limited	Rejected
LangGraph	State-machine graph	Indirect	Possible	Rejected

Table 2. Multi-agent frameworks compared.

I chose CrewAI because the travel domain decomposes naturally into roles (flights, hotels, attractions), it ships a documented MCP adapter, and it embeds in Streamlit in a few dozen lines of code. AutoGen's free-form style would re-introduce the very semantic-drift problem I am trying to avoid.

2.3 Tool and messaging protocols

Tool calling has started to follow the path of JSON-schema-validated function calls, which OpenAI popularized in 2023 and are now adopted by all vendors (MCP, Anthropic, 2024). There is no consensus with regard to inter-agent messaging. Park et al. (2023) observed that the inter-agent communication was in free text and it was brittle. The classical FIPA Communicative Acts Library (FIPA, 2002) is the most complete, published list of typed messages sent by agents, and directly motivates the use of the small A2A envelope I describe in §3.6.

2.4 AI travel planning systems

The idea of academic prototypes in the field of itinerary planning dates back to at least 10 years. A constraint solver over a closed database is used by Lim et al. (2019) and they report 78% user satisfaction and no live booking. The most recent benchmarking is the TravelPlanner benchmark (Xie et al., 2024), which provides a clean evaluation set but does not suggest a working system, and has the best baseline of 0.6% final-pass. Commercial offerings such as Kayak's AI search are closed-source. As far as I know, no published or open-sourced system has been found that achieves all three of the following at once: a multi-agent decomposition, a schema-validated tool layer and live booking-API integration.

2.5 Conceptual model and gap

When viewed as a whole, the literature identifies three constant failure scenarios: context bloat, protocol fragility, and semantic drift, and two recent technical opportunities: multi-agent frameworks are well developed, and there are tool protocols that are also standardized. My concept is that if each agent has one narrow domain, each tool call is schema validated, and each inter-agent message is typed, the three failure modes should be structurally avoided in preference to being patched. It's not one thing that is new about this project, it's all three things being applied in a disciplined manner on a real live-API domain.

3. Research Methodology

This is the longest as methodology contributes 30% of the marks and the value of the project lies in decisions made about design. Every step I explain the options that I was considering, why I chose the option and what its drawbacks were.

3.1 Research paradigm

I'm going to use the Design Science Research (DSR) paradigm of Hevner et al. (2004) to solve a problem, create an artefact, and then evaluate it. I thought about using an empirical approach to comparing off-the-shelf assistants and a formal modelling approach - neither would have got me working software. DSR fits since the contribution is an empirical runnable system.

3.2 System design

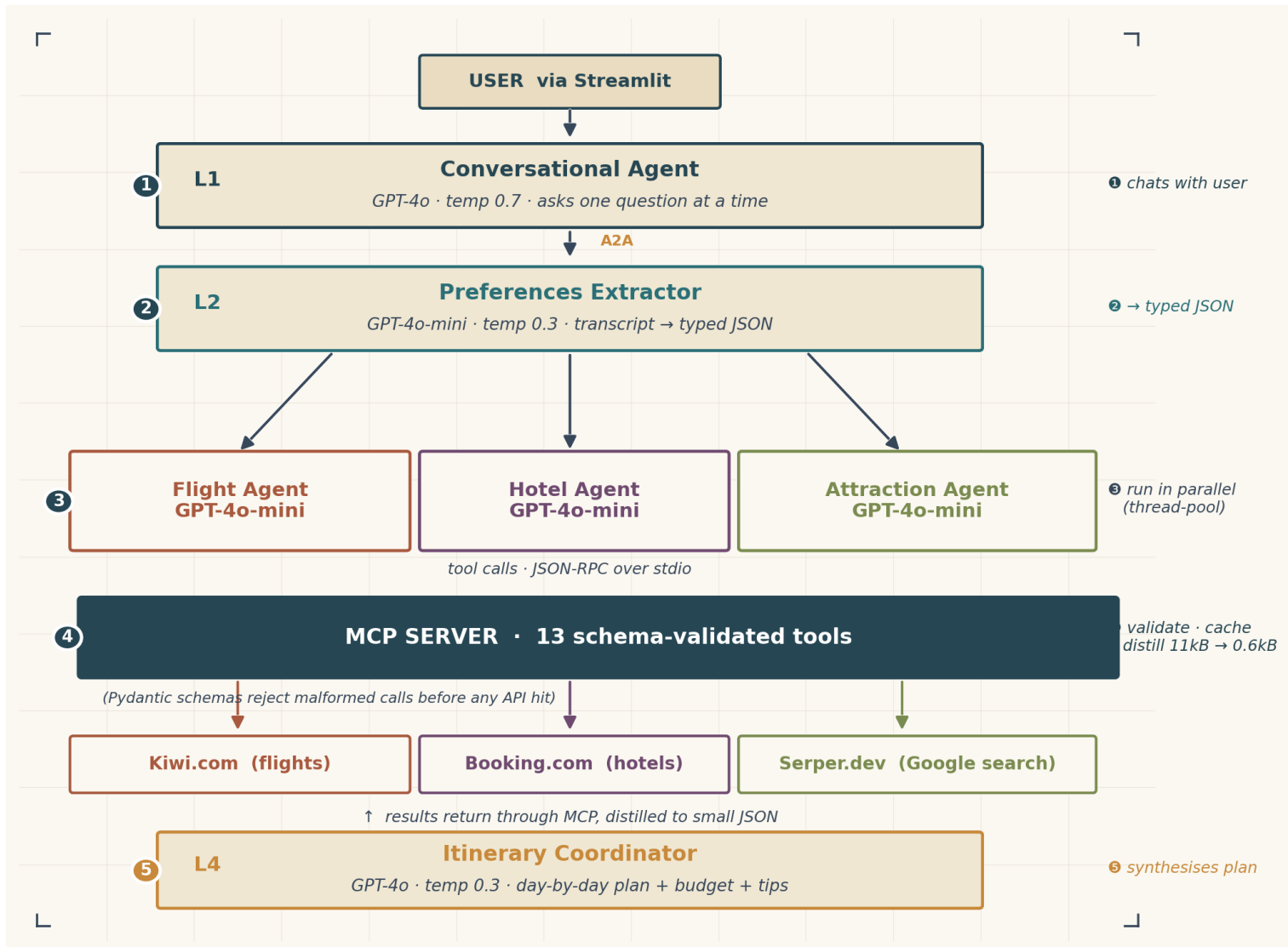


Figure 1. System architecture: four layers, six agents, one MCP server, three live APIs.

The system is divided into four horizontal layers, as illustrated in Figure 1. Layer 1 interacts with the user. Layer 2 morphs the dialogue into typed JSON. There are three search agents running in parallel in Layer 3, which call APIs via the MCP server. The day-by-day plan is synthesized on Layer 4. Agents will never send free text, and any inter-agent message is sent via the A2A envelope shown in Figure 4.

3.3 Agents and their responsibilities

Agent	Input	Output	Stops when
Conversational	User free-text	Preference transcript	All fields collected
Preferences Extractor	Transcript	Typed JSON (Pydantic)	Schema validates
Flight Agent	Typed JSON	Top 3 flight options	≥3 options or 4 calls
Hotel Agent	Typed JSON	1 hotel per leg	1 hotel per leg chosen
Attraction Agent	Typed JSON + cities	5 attractions + 2 restaurants	Quotas met or 4 calls
Coordinator	Three RESPONSE messages	Day-by-day plan	Plan passes checks

Table 3. Agent responsibilities (each unit-testable in isolation).

3.4 Internal React loop

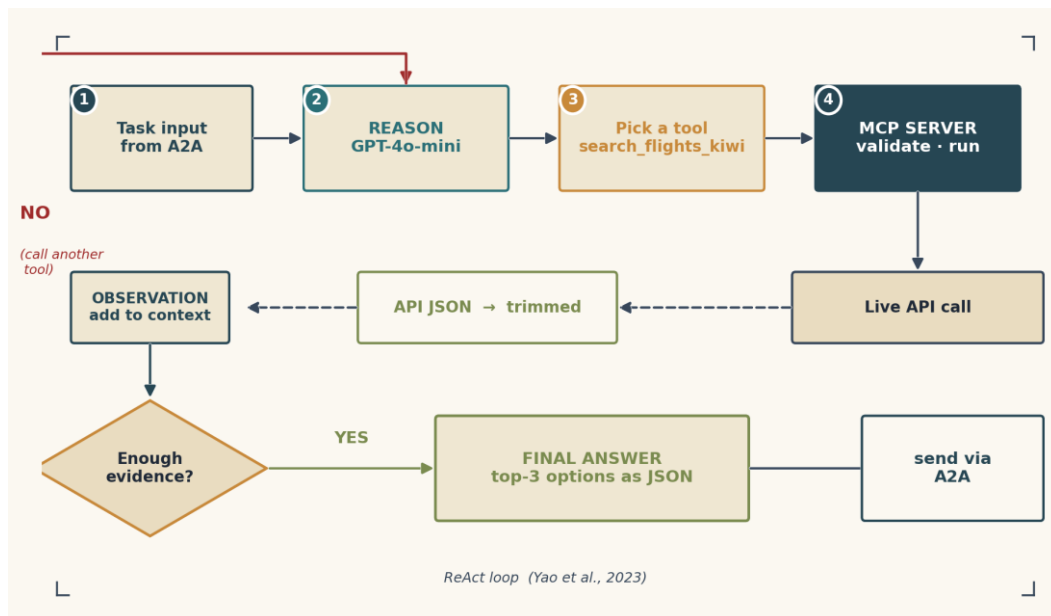


Figure 2. Internal React loop inside a specialist agent.

The search agents execute a loop, similar to a React loop (Yao et al., 2023): observe, select a tool, invoke the tool via MCP, extract the distilled output, and determine if to halt. For instance, if the cheapest result exceeds the budget, the Flight Agent calls back the

search_flights_kiwi flight search with wider time parameters. It returns the top three choices after a maximum of four tool calls; not the 12 kB API response.

3.5 MCP server lifecycle

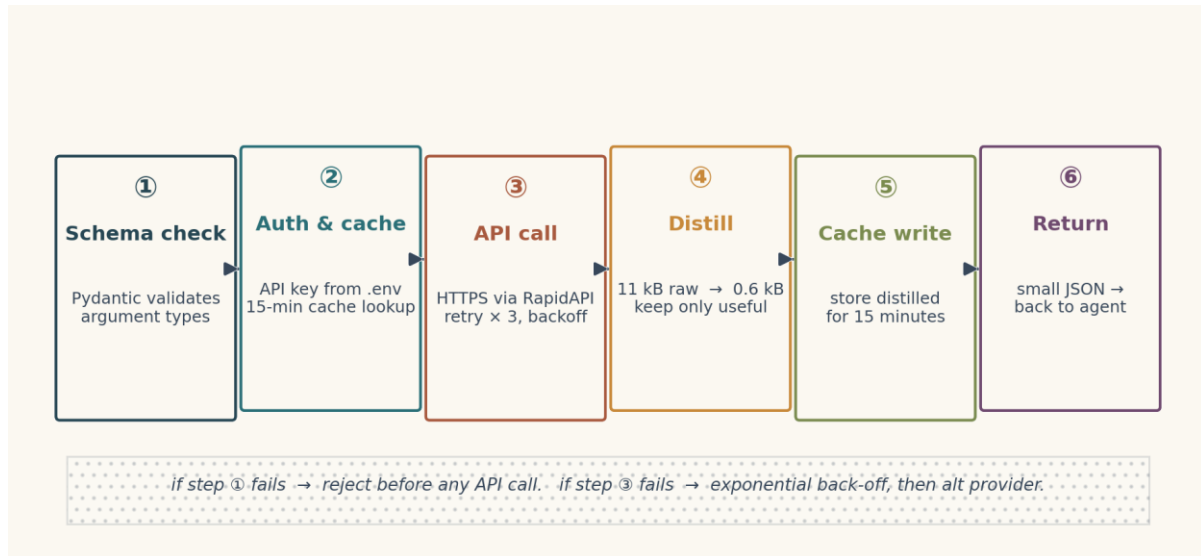


Figure 3. MCP server request lifecycle.

The single most important design element is the MCP server. It is a Python sub process connected via JSON-RPC over stdio, and offers thirteen tools (Table 4). Each tool description has corresponding Pydantic input schema that prevents a tool call from saying anything bad about an external API. This is the six-stage lifecycle in figure 3, which includes schema check, auth and cache lookup, live API call with retry and backoff, distillation, cache write and return.

#	Tool	API	Purpose
1	search_flights_kiwi	Kiwi.com	Flight search by city / date
2	search_cheap_flights	Kiwi.com	Flexible-date flight search
3	search_hotels_comprehensive	Booking.com	Hotel search with reviews
4	search_hotel_destination	Booking.com	Resolve city to dest_id
5	search_hotels_by_destination	Booking.com	Hotel search by dest_id
6	get_hotel_reviews	Booking.com	Pull review scores

7	get_attractions_near_hotel	Booking.com	POIs near a hotel
8	search_accommodations_with_location	Booking.com	GPS-based search
9	search_car_rentals	Booking.com	Car-rental search
10	search_internet	Serper.dev	Google web search
11	search_attractions	Serper.dev	Attraction search
12	search_restaurants	Serper.dev	Restaurant search
13	calculate	Local	Safe arithmetic

Table 4. Tools exposed by the MCP server.

3.6 A2A protocol

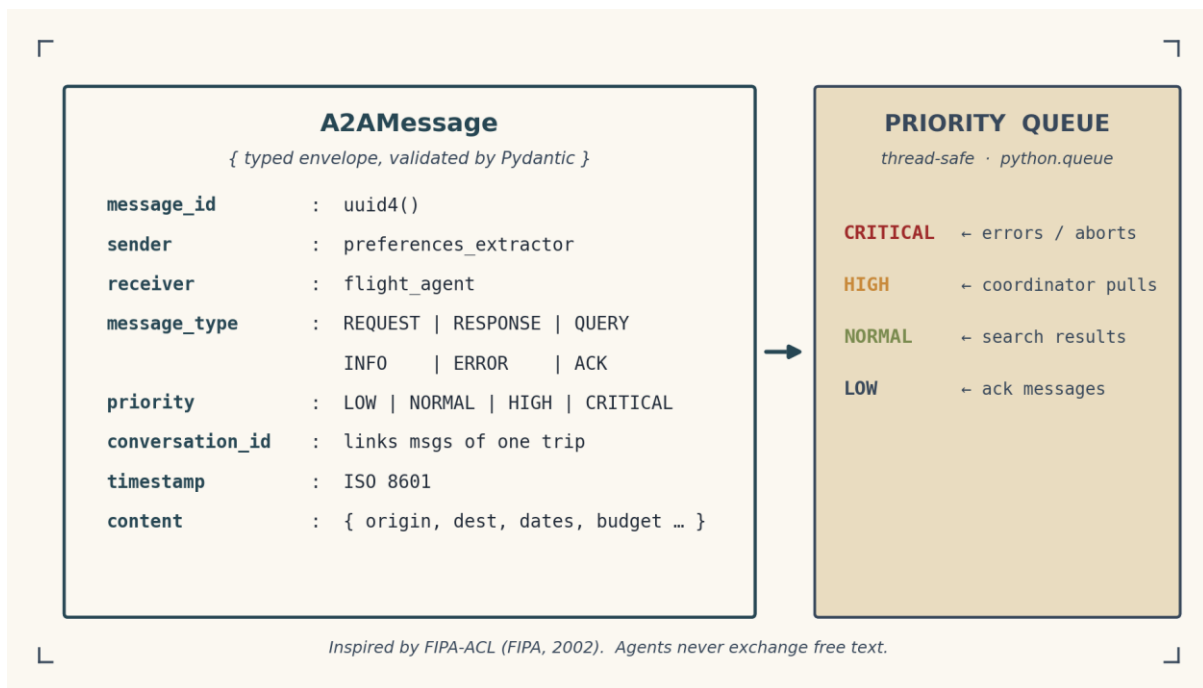


Figure 4. A2A message envelope and priority queue.

Each inter-agent message has an id, a sender, a receiver, one of six types (REQUEST, RESPONSE, QUERY, INFO, ERROR, ACK), a priority, a conversation id (which can be used to correlate the messages of a given trip) and a timestamp, and a typed content payload. A thread-safe priority queue provides delivery of messages in the order of the priority, with CRITICAL at the top, followed by HIGH, followed by NORMAL, followed by LOW.

3.7 End-to-end sequence and data flow

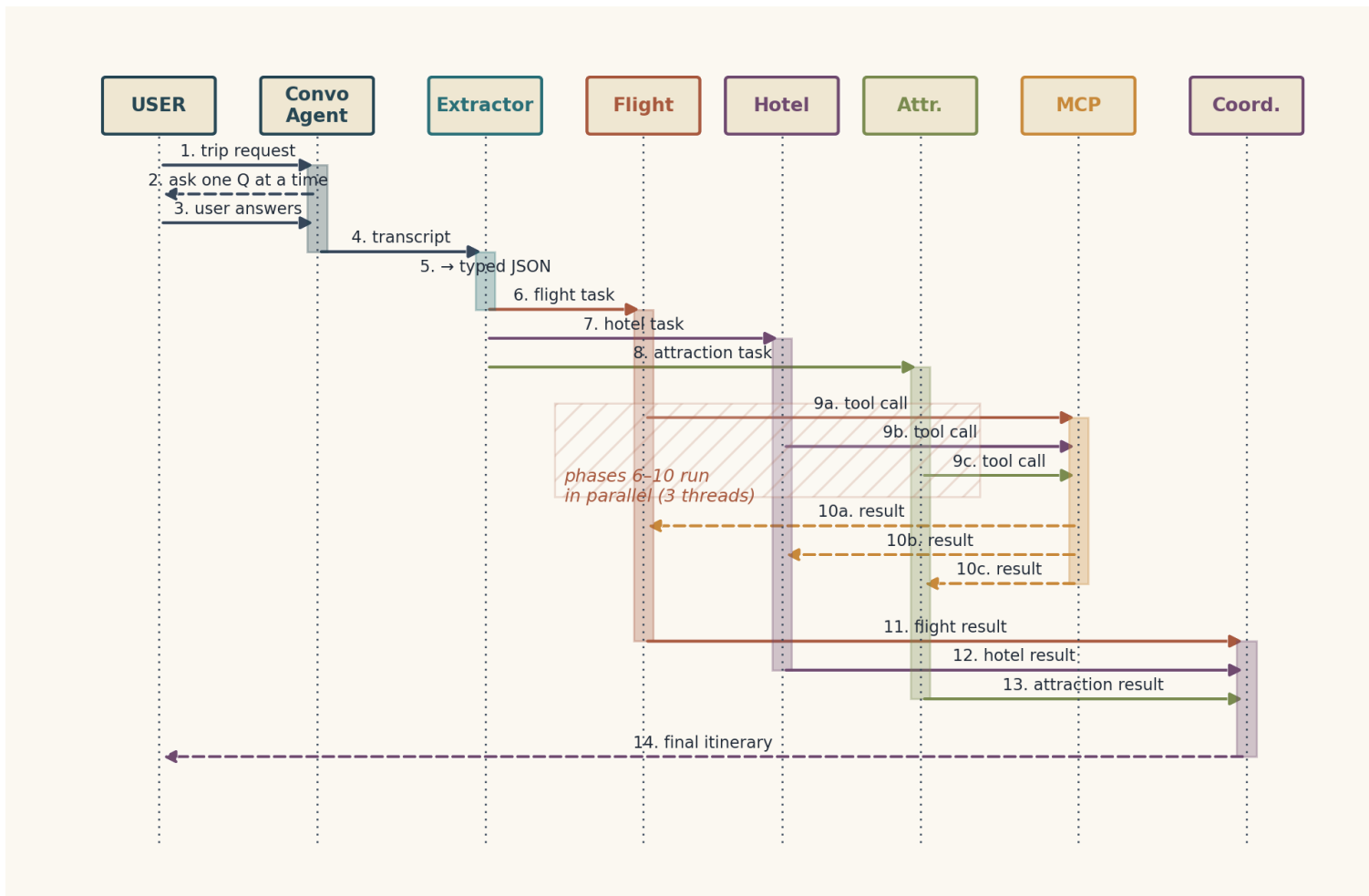


Figure 5. End-to-end sequence for one trip request.

Each search agent is executed in parallel via a thread-pool executor (Figure 5), and the outbound HTTPS requests are executed concurrently in the MCP server, also in asyncio. The Coordinator just fires when all three responses are received, this way the final synthesis is deterministic. Figure 6 reinterprets the system as a Level-1 data-flow diagram, and exposes the persisted artefacts (a transcript log for traceability, a preferences JSON for reproduction).

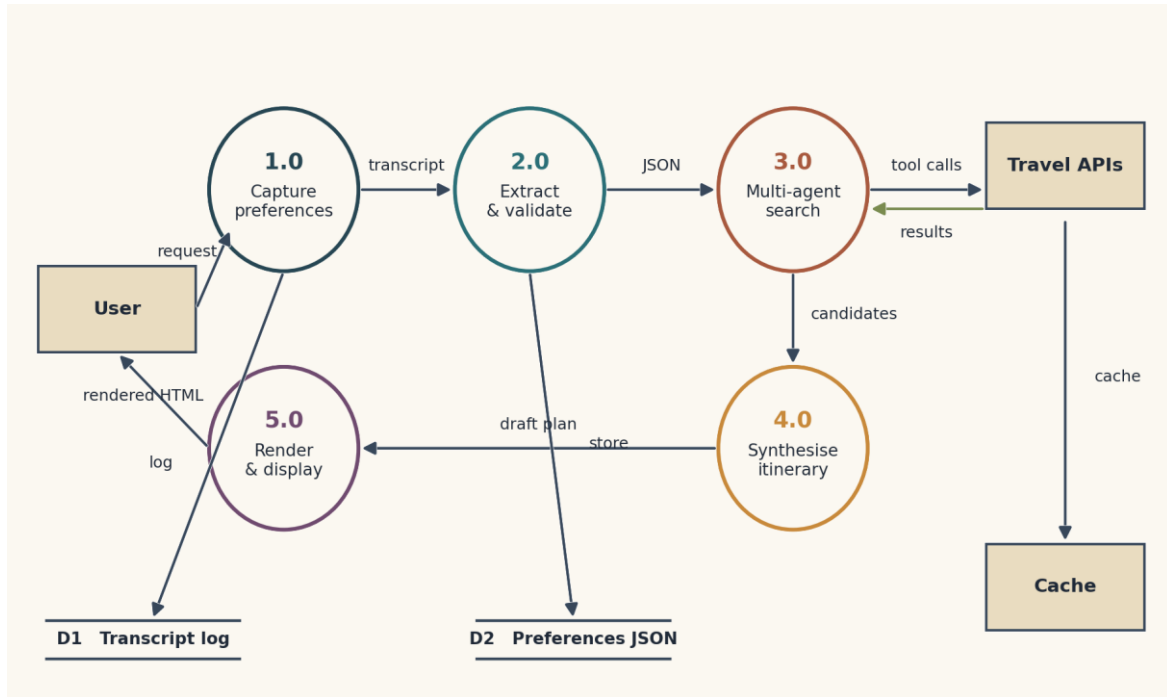


Figure 6. Level-1 data-flow diagram.

3.8 Models, APIs and cost

The structured extraction and search agents are based on the GPT-4o-mini (Table 5), a lower cost model, while the other agents, the Coordinator and the Conversational Agent, rely on the more expensive GPT-4o model. This split is the predominant one that I am using to keep my cost-per-itinerary target lower than USD 1. The MCP server receives three live APIs: Kiwi.com - flights, Booking.com - hotels via RapidAPI and Serper.dev - Google search. All three provide free accounts that are sufficient for development.

Agent	Model	Temp	Why
Conversational	GPT-4o	0.7	Warm, natural dialogue
Extractor	GPT-4o-mini	0.3	Pure structured extraction
Flight	GPT-4o-mini	0.3	Few-tool deterministic search
Hotel	GPT-4o-mini	0.3	Few-tool deterministic search
Attraction	GPT-4o-mini	0.3	Few-tool deterministic search
Coordinator	GPT-4o	0.3	Multi-step synthesis

Table 5. Model and temperature per agent.

3.9 Alternatives considered

Decision	Selected	Alternatives considered	Why selected
Agent framework	CrewAI	AutoGen, MetaGPT, LangGraph	Role-based fit, MCP adapter, low learning curve
Tool protocol	MCP (JSON-RPC)	Vendor function-calling only, custom REST	Strict schema, vendor-neutral
LLM provider	OpenAI 4o / 4o-mini	Anthropic Claude 3.5, Google Gemini 1.5	Best price/performance for tool calling
Flight API	Kiwi.com (RapidAPI)	Skyscanner, Amadeus self-service	Free tier, flexible-date support
Hotel API	Booking.com (RapidAPI)	Hotels.com, Expedia	Single endpoint covers hotels + reviews
Search API	Serper.dev	Bing, Brave, scraping	Cheapest reliable Google results
Web UI	Streamlit	Gradio, Flask, React + FastAPI	Fastest path to a viva-ready demo
Inter-agent comms	Custom A2A envelope	Free-text, raw OpenAI threads	Prevents semantic drift; testable

Table 6. Decisions, alternatives and rationale.

3.10 Evaluation plan

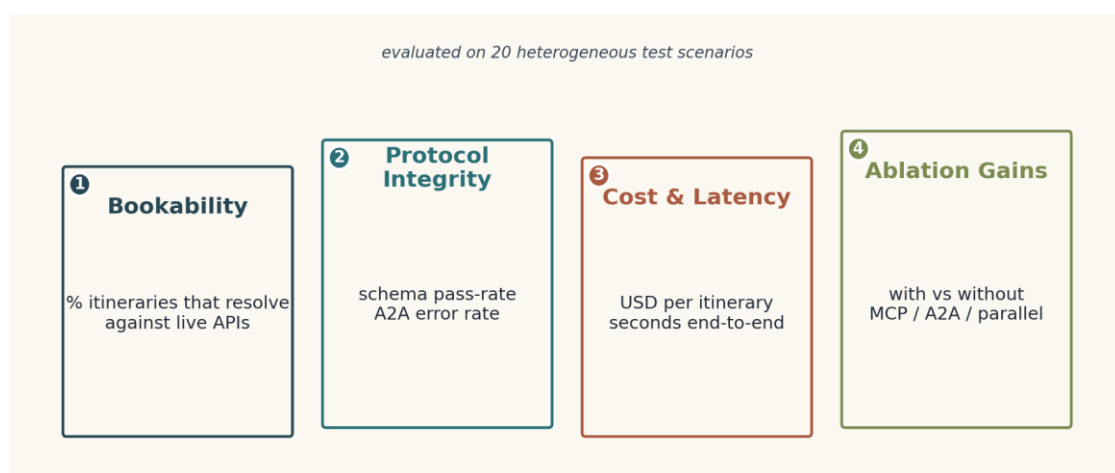


Figure 7. Four-pillar evaluation framework.

There will be a set of 20 heterogeneous test scenarios designed beforehand: short city breaks (Lahore-Istanbul, 4 nights, \$800), long-haul family trips (Lahore-Tokyo, 8 nights, \$3,000), beach and city combinations (Lahore-Bangkok-Phuket, 10 nights, \$3,000), tight-budget edge cases (Karachi-Bangkok, 5 nights, \$700), and obvious failure cases (impossible budgets) where it is clearly an error. Four metrics are presented: bookability (% of itineraries that still complete at re-query), protocol integrity (MCP schema pass-rate, A2A error rate), cost and latency, and ablation gains. Bookable: $\geq 80\%$, Schema Pass rate: 100%, A2A error rate: $< 1\%$, Latency (ms): < 4 , Improvement over a single-LLM model: $\geq 25\%$.

3.11 Risk register

#	Risk	L	I	Mitigation
R1	RapidAPI rate limit during evaluation	M	M	15-min cache; alt provider
R2	OpenAI prices change mid-project	L	M	config.yaml swap-in
R3	Single-LLM baseline harder than expected	M	M	Two simpler baselines as fallback
R4	Booking.com terms-of-service change	L	H	Read-only; fallback to Hotels.com
R5	Time over-run on a component	M	M	2-week buffer in Gantt
R6	Hardware failure	L	H	Daily push to GitHub + OneDrive

Table 7. Risk register. L = Likelihood, I = Impact (L/M/H).

3.12 Resources

Category	Resource	Cost
Compute	Personal laptop, 16 GB RAM, no GPU required	Owned
LLM API	OpenAI 4o + 4o-mini (≈600 calls)	≈ £20
Travel APIs	Kiwi, Booking, Serper free tiers	£0
Code hosting	GitHub free tier	£0
Reference manager	Zotero (BCU Harvard style)	£0
Contingency	Paid RapidAPI tier if needed	≈ £10

Table 8. Indicative resources (GBP).

3.13 Ethics and limitations

Ethics first. Only RapidAPI free-tier endpoints will be used, well within their monthly limits, and no scraping is involved. The Streamlit demo stores neither personally identifiable information nor payment data; the local transcript log is anonymized before any analysis. The UI carries a clear disclaimer that the system is a planning aid and not a booking system. I will follow the BCS Code of Conduct (BCS, 2022) and the BCU academic-integrity guidance throughout. I will confirm at the W1 supervisor check-in that no full ethics application is required, since there is no human-subject research.

Limitations are also worth being honest about. Free RapidAPI tiers have rate limits — the cache plus a fallback tool addresses this. OpenAI pricing may shift mid-project — model assignment lives in a single YAML file. Evaluation relies on twenty scripted scenarios rather than a real user study, which limits external validity; the dissertation will treat a live user study as future work. Live API availability is outside my control, so every evaluation run will record raw API responses for offline replay. Finally, I am a single developer with no formal multi-agent experience, which is precisely why the design is deliberately small (six agents, thirteen tools) and why I have agreed weekly half-hour supervisor check-ins.

4. Project Schedule

The proposal is submitted on 22 June 2026, leaving exactly eleven weeks before the dissertation deadline of 7 September 2026. I plan a weekly cadence: each Monday a half-hour supervisor check-in to review progress, agree blockers and adjust the plan if needed. Figure 8 shows the Gantt chart for those eleven weeks; dissertation write-up runs in parallel from week 5 so no time is lost at the end.

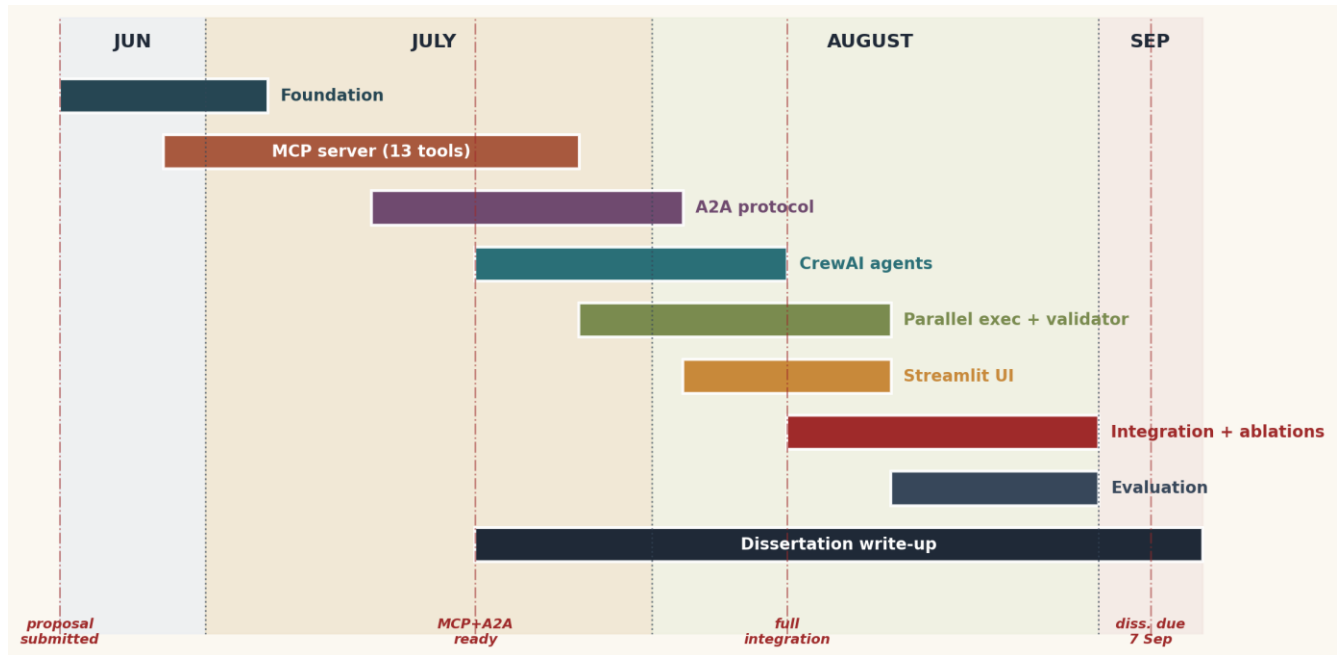


Figure 8. Gantt chart, 23 June – 7 September 2026 (11 weeks).

#	Task	Weeks	Milestone
T1	Foundation and architecture refresh	W1–W2	Updated design doc
T2	MCP server: 13 tools + Pydantic schemas	W2–W5	Working MCP server
T3	A2A protocol: envelope + priority queue	W4–W6	Library + tests
T4	Six CrewAI agents wired to tools	W5–W7	End-to-end response
T5	Parallel execution + budget validator	W6–W8	<4-min latency
T6	Streamlit web UI	W7–W8	Working demo
T7	Integration + ablations	W8–W10	Ablation results

T8	Evaluation across 20 scenarios	W9–W10	Evaluation report
T9	Dissertation write-up + viva prep (parallel)	W5–W11	Submitted 7 Sep

Table 9. Tasks, weeks and milestones.

Two weeks of internal buffer are built in: T7 and T8 do not start until the core system is finished in W7, and T9 absorbs any small slip. If a major component slips by more than a week my contingency is to drop `search_car_rentals` and `get_attractions_near_hotel` from the MCP server, since neither is on the critical path of any evaluation scenario.

5. References

References follow the BCU Harvard convention.

1. Anthropic (2024) Model Context Protocol Specification. Available at: <https://modelcontextprotocol.io> (Accessed: 1 June 2026).
2. BCS (2022) BCS Code of Conduct for Members. Swindon: British Computer Society.
3. CrewAI (2024) CrewAI Documentation: Agents, Tasks and Crews. Available at: <https://docs.crewai.com> (Accessed: 5 June 2026).
4. FIPA (2002) Foundation for Intelligent Physical Agents — Communicative Act Library Specification (FIPA-ACL). Geneva: FIPA.
5. Hevner, A. R., March, S. T., Park, J. and Ram, S. (2004) ‘Design Science in Information Systems Research’, *MIS Quarterly*, 28(1), pp. 75–105.
6. Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C. and Schmidhuber, J. (2024) ‘MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework’, *ICLR 2024*.
7. Park, J. S., O’Brien, J., Cai, C. J., Morris, M. R., Liang, P. and Bernstein, M. S. (2023) ‘Generative Agents: Interactive Simulacra of Human Behavior’, in *Proceedings of UIST 2023*.
8. Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z. and Wen, J. (2024) ‘A Survey on Large Language Model Based Autonomous Agents’, *Frontiers of Computer Science*, 18 (arXiv:2308.11432).
9. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, X. and Wang, C. (2023) ‘AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation’, *arXiv:2308.08155*.

10. Xi, Z., Chen, W., Guo, X., He, W. and Gui, T. et al. (2023) 'The Rise and Potential of Large Language Model Based Agents: A Survey', arXiv:2309.07864.